Review lab app py test app py

ID: local-a7749a5d95c6ab0a63ce432e7b4f8598b563fbe1eb085e3d54dcaeb8e30fe694

Model: upstage/solar-mini4:free **Started:** 2026-11-08 21:21:54

System Prompt (Persona)

USER 2026-11-08 21:22:01

Revise lab/app.py e lab/test_app.py. Este é o mesmo laboratório local de SQLite, após a correção. Mostre como os parâmetros separam os valores da instrução SQL e explique quais testes verificam que o login legítimo funciona, a senha incorreta é rejeitada e a tentativa original de injeção perde o efeito. Separe a revisão do código do resultado da execução. Faça a revisão sem editar arquivos.

ASSISTANT 2026-11-08 21:22:08

TOOL 2026-11-08 21:22:20

TOOL 2026-11-08 21:22:25

ASSISTANT 2026-11-08 21:22:28

TOOL 2026-11-08 21:22:31

TOOL 2026-11-08 21:22:31

TOOL 2026-11-08 21:22:31

TOOL 2026-11-08 21:22:31

ASSISTANT 2026-11-08 21:22:45

TOOL 2026-11-08 21:22:47

ASSISTANT 2026-11-08 21:23:38

TOOL 2026-11-08 21:23:41

ASSISTANT 2026-11-08 21:24:09

Aqui está a revisão. Não editei nenhum arquivo — só lei e testei.

Resultado da execução

Do diretório raiz, o comando oficial do README:

```
python -m unittest discover -s lab -v
```

```
test_credenciais_validas (test_app.TesteLogin.test_credenciais_validas)
... ok
test_reproducao_e_mitigacao_da_injecao
(test_app.TesteLogin.test_reproducao_e_mitigacao_da_injecao) ... ok
test_senha_invalida (test_app.TesteLogin.test_senha_invalida) ... ok
```

```
----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

Comportamento do fluxo antes/depois da correção (mesmo banco em memória, mesmo payload):

```
[login legítimo]
  SQL: 'SELECT 1 FROM usuarios WHERE nome = ? AND senha = ?'
  Parâmetros: ('aluna', 'senha-lab')
  autenticar → True

[senha incorreta]
  SQL: 'SELECT 1 FROM usuarios WHERE nome = ? AND senha = ?'
  Parâmetros: ('aluna', 'errada')
  autenticar → False

[payload injeção]
  SQL: 'SELECT 1 FROM usuarios WHERE nome = ? AND senha = ?'
  Parâmetros: ("aluna' -- ", 'errada')
  autenticar → False
```

Revisão de código

lab/app.py

Parametrização (o ponto central). A consulta é:

```python
consulta = "SELECT 1 FROM usuarios WHERE nome = ? AND senha =
?"
return banco.execute(consulta, (nome, senha)).fetchone() is not None
```

A instrução SQL é uma string fixa: a tabela, as colunas e a lógica do

`WHERE` não mudam nunca. Os dados do usuário (`nome` e `senha`) entram apenas como **valores de bind**, num tuplo separado do texto da consulta. O sqlite3 substitui cada `?` por um literal, um a um. Por isso:

- A instrução fica sempre: `SELECT 1 FROM usuarios WHERE nome = ? AND senha = ?`
- O valor de `nome` vira um literal `"aluna' -- "` e o `'` **Não fecha nenhuma string** dentro do SQL, porque o `'` não está no texto da consulta.

Comparação com o código antes da correção (`lab/vulneravel.py`): ali a consulta era uma f-string que injetava os dados direto no texto:

```python
f"SELECT 1 FROM usuarios WHERE nome = '{nome}' AND senha = '{senha}'"
```

Com `nome = "aluna' -- "` o SQL vira `SELECT 1 FROM usuarios WHERE nome = 'aluna' -- ' AND senha = '...'`, e o `-- ` faz o restante da linha ser comentado, validando só o nome. Na versão corrigida não há essa fusão entre dados e instrução; o payload vira um valor de texto que não corresponde a nenhum usuário.

Outros pontos:
- `closing(criar_banco())` fecha a conexão em memória corretamente.
- `getpass()` para senha evita que o valor seja impresso na tela.
- `criar_banco()` insere `("aluna", "senha-lab")`; `main()` usa `argparse` com descrição, mas nenhum argumento — inofensivo, um "leve desnecessário".
- Nada de crítico nesta versão.

lab/test_app.py

Cobre exatamente os três cenários pedidos:

1. **Login legítimo funciona** — `test_credenciais_validas`: recria o banco (que contém `aluna`/`senha-lab`) e chama `autenticar(banco, "aluna", "senha-lab")`, esperando `True`. Confirma que a correção não

quebrou o fluxo normal.
2. **Senha incorreta é rejeitada** — `test_senha_invalida`: mesmo com o nome certo, `autenticar(banco, "aluna", "errada")` espera `False`.
3. **O payload de injeção perde o efeito** — `test_reproducao_e_mitigacao_da_injecao`: usa o mesmo banco e o payload original `"aluna' -- "` como nome, senha `"errada"`:
   - `self.assertTrue(autenticar_vulneravel(banco, entrada, "errada"))` — reproduz a falha na versão original para mostrar o que seria o ataque (e que o teste está comparando contra o caminho antigo).
   - `self.assertFalse(autenticar(banco, entrada, "errada"))` — a versão corrigida rejeita o mesmo payload; o `aluna' -- ` vira literal sem corresponder a nenhum registro, e a senha errada, isolada, já faria `False`.

A divisão em métodos é limpa; cada teste cria seu próprio banco no `with closing(...)`, então não há estado cru entre eles. O import de `app` e de `vulneravel` depende da raiz estar no caminho de busca, o que ocorre com `python -m unittest` pela raiz — conforme o README.

Conclusão: revisão e execução confirmam que a correção está no lugar e os testes cobrem o que se pede. Os dois arquivos não precisam de alteração.

Reasoning